

Phase 2

Tech Stack:

- React
- Python
- Sci-kit learn
- FastAPI
- Google Cloud
- Google Firebase
- Gemini API

Frontend: React, Vercel

Backend: Python, Sci-kit Learn, FastAPI

Cloud: Google Cloud Run

Database: Firebase (Firestore + Auth)

AI: Gemini API

Machine Learning Model

The main aim of the machine learning model is to predict the probability that a patient is at a risk of health deterioration.

Input → Patient data

Output → Risk score (1-3) + Risk category (Low / medium / high)

Fetch dataset from : <https://physionet.org/>

Search for:

- MIMIC - IV
- MIMIC - III
- Sepsis Challenge
- eICU dataset

System Flow:

1. Patient enters data
2. Data preprocessing
3. Feature Engineering
4. Random forest predicts probability

5. Safety rule check
6. Risk category (1-3)
7. Safety rule check
8. Medical attention remark
9. Return result

Main Algorithm: Random Forest Classifier

Baseline: Logistic Regression

Advanced Boost: XGBoost

Model Features:

- Core vitals (MANDATORY)
- Blood / Lab Indicators
- Medical history (Risk Amplifiers)
- Symptom Features
- Trend Features

Core: HR, SBP, DBP, SpO₂, RR, Temp

History: Diabetes, Hypertension, Heart Disease

Symptoms: ChestPain, Breathlessness, Fever

Trend: HR_change, SpO₂_change

How does the patient enter the vitals??

In hospitals:

- Nurses measures the vitals
- They manually enter into the system
- Doctor reviews the dashboards

At home

Patients use:

- Pulse oximeter
- BP Monitor
- Thermometer
- Smartwatch

How to make it feel real on website:

1 Add Normal Range

Under each field:

SpO₂ (Normal: 95–100%)

2 Add Input Validation

Reject unrealistic numbers.

3 Add Time-Based Entry

“Current reading”

“Previous reading (3 hrs ago)”

Patients and nurses enters value from:

- Standard medical devices
- Hospital equipment
- Home monitoring tools

Our System:

- Validates

- Analyzes
- Predicts
- Explains

How to make the model more accurate?

1. Strong signal in data
2. Clean Pre-processing
3. Proper feature engineering
4. Good model choice
5. Correct evaluation
6. No leakage
7. Balanced Dataset

How to push to 90-95% Accuracy

1 Get a High-Quality Dataset

This is the biggest factor.

If the dataset:

- Has clear deterioration labels
- Has time-series vitals
- Has minimal missing data

Our chances increase massively.

Bad data = impossible to hit 90%.

2 Engineer Powerful Features

Raw vitals are weak.

Trend features are powerful.

Create:

- HR change rate
- SpO₂ decline percentage
- BP variability
- Rolling averages
- Vital instability score

Add interaction terms:

- Age × HR
- Diabetes × BP

Feature engineering alone can boost 10–20%.

3 Handle Class Imbalance Properly

Use:

- SMOTE
- class_weight="balanced"
- Stratified train-test split

If high-risk class is rare and you don't fix imbalance → accuracy becomes fake.

4 Use Strong Models

For tabular medical data:

Best options:

- Random Forest
- XGBoost
- Gradient Boosting
- LightGBM

These outperform logistic regression.

If allowed, XGBoost is extremely powerful.

5 Hyperparameter Tuning (Mandatory)

Use: GridSearchCV or RandomizedSearchCV

Tune:

- depth
- learning rate
- estimators
- regularization

This can improve 3–8%.

6 Cross Validation

Use 5-fold or 10-fold cross validation.

Avoid single splits.

This stabilizes performance.

7 Avoid Overfitting

If you see:

Train accuracy = 98%

Test accuracy = 82%

You're overfitting.

Control using: Max_depth, Regularization, pruning